

Vitaay Escrow Payment System

Enterprise Product & UX Specification

Document Version	2.0
Status	Draft for Cross-Functional Review
Prepared For	Product, Engineering, UI/UX Design, Compliance, QA, Business Stakeholders
Classification	Internal — Confidential
Source	Vitaay Escrow Payment System PRD v1.0 (June 2026)

Vitaay Escrow Payment System — Enterprise Product & UX Specification

Version History

Table of Contents

1. Executive Summary
2. Product Vision, Goals & KPIs
 - 2.1 Vision Statement
 - 2.2 Business Goals
 - 2.3 Success Criteria (Launch Definition of Done)
 - 2.4 KPIs
3. The Problem
 - 3.1 Without a Payment System
 - 3.2 Business Impact
 - 3.3 Why This Is Urgent Now
4. Solution Overview
5. Users & Roles
 - 5.1 Role Capability Matrix
6. Payment Architecture
 - 6.1 System Components
 - 6.2 Stripe Building Blocks
 - 6.3 Escrow Lifecycle (Architectural View)
 - 6.4 Internal Services & Data Flow
 - 6.5 Webhook Event Catalogue
 - 6.6 Sequence Diagram — Fund → Hold
7. Complete Payment Flows (Happy Path + Every Edge Case)
 - 7.1 Happy Path — Full Campaign Payment
 - 7.2 Payment Failure (Card Declined at Funding)
 - 7.3 Refund (Brand-Initiated, Escrow Held)
 - 7.4 Chargeback / Dispute (Stripe-Initiated)
 - 7.5 Retry After Failure
 - 7.6 Cancel (Brand Abandons Checkout Before Confirming)
 - 7.7 Duplicate Payment Prevention (Double-Click / Double-Submit)
 - 7.8 Network Failure Mid-Payment (Client Loses Connection After Charge, Before Response)
 - 7.9 Webhook Delay
 - 7.10 Webhook Duplicate Delivery
 - 7.11 Timeout (Stripe API Slow/Unresponsive)
 - 7.12 Session Expiration Mid-Checkout
8. Escrow Status State Machine
 - 8.1 States
 - 8.2 State Diagram
 - 8.3 Guard Rules
9. UI/UX Specification (20 Screens)
 - 9.1 Screen 1 — Creator Payment Onboarding
 - 9.2 Screen 2 — Brand Checkout / Fund Escrow
 - 9.3 Screen 3 — Stripe Payment (Embedded Element State)
 - 9.4 Screen 4 — Processing Screen
 - 9.5 Screen 5 — Success Screen
 - 9.6 Screen 6 — Failure Screen
 - 9.7 Screen 7 — Escrow Status Card

- 9.8 Screen 8 — Release Payment Confirmation (Modal)
- 9.9 Screen 9 — Refund Confirmation (Modal)
- 9.10 Screen 10 — Transaction History
- 9.11 Screen 11 — Creator Earnings Dashboard
- 9.12 Screen 12 — Tip Button + Modal
- 9.13 Screen 13 — Brand Dashboard (Payments Widget)
- 9.14 Screen 14 — Notification Center
- 9.15 Screen 15 — Payment Settings
- 9.16 Screen 16 — Bank Settings (Stripe-Hosted, External)
- 9.17 Screen 17 — Admin Escrow Management
- 9.18 Screen 18 — Audit Logs
- 9.19 Screen 19 — Support Screen
- 9.20 Screen 20 — Dispute Center
- 10. Worldwide UX Benchmarking
- 11. Compliance
 - 11.1 PCI-DSS
 - 11.2 GDPR (and UK GDPR)
 - 11.3 PSD2 / SCA (Strong Customer Authentication)
 - 11.4 3D Secure
 - 11.5 AML (Anti-Money Laundering)
 - 11.6 KYC (Know Your Customer)
 - 11.7 OFAC / Sanctions Screening
 - 11.8 RBI Guidelines (India)
 - 11.9 Visa / Mastercard Network Rules
 - 11.10 Chargeback Guidelines & Refund Policies
 - 11.11 Consumer Protection
 - 11.12 Accessibility (WCAG 2.2 AA)
- 12. Security
 - 12.1 Card Security & Tokenization
 - 12.2 Idempotency
 - 12.3 Server-Side Fee Calculation
 - 12.4 Webhook Signature Verification
 - 12.5 Idempotency & Replay Protection
 - 12.6 Encryption
 - 12.7 Rate Limiting
 - 12.8 CSRF / XSS / Clickjacking
 - 12.9 Fraud Detection & Velocity Checks
 - 12.10 Device Fingerprinting & Suspicious Activity
 - 12.11 Logging, Audit Trail, Secrets, Key Rotation, Session Security
- 13. UI Checklists
 - 13.1 Checkout Checklist
 - 13.2 Success Checklist
 - 13.3 Failure Checklist
 - 13.4 Refund Checklist
 - 13.5 Escrow Checklist
 - 13.6 Creator Dashboard Checklist
 - 13.7 Brand Dashboard Checklist
 - 13.8 Admin Checklist
 - 13.9 Settings Checklist
 - 13.10 Accessibility Checklist
 - 13.11 Mobile / Tablet / Desktop Checklist
 - 13.12 Dark Mode Checklist
 - 13.13 Localization / Currency / Timezone / Language Checklist

- 14. Error Catalogue
- 15. Notifications Matrix
- 16. API ↔ UI Mapping
- 17. QA Test Plan
 - 17.1 Positive Scenarios
 - 17.2 Negative Scenarios
 - 17.3 Boundary Scenarios
 - 17.4 Security Test Scenarios
 - 17.5 Accessibility Scenarios
 - 17.6 Localization Scenarios
 - 17.7 Performance / Load Scenarios
 - 17.8 Network Interruption / Retry / Duplicate Click / Session Expiry / Refresh Scenarios
- 18. Analytics
 - 18.1 Core Events
 - 18.2 Funnel Tracking
 - 18.3 North Star & Supporting KPIs
- 19. Admin Requirements
 - 19.1 Admin Dashboard Capabilities
 - 19.2 Audit Log Requirements
 - 19.3 Search & Filters
- 20. Engineering Status
- 21. Go-Live Readiness
- 22. Risks and Mitigations
- 23. Future Roadmap (Phase 2+)
- 24. Glossary
- 25. Appendices
 - Appendix A — Fee Breakdown Example (from original PRD, retained)
 - Appendix B — Screen Inventory Summary
 - Appendix C — Original Payment Flow Diagrams (Preserved for History)
 - Appendix D — Notes on Source Document Fidelity

Vitaay Escrow Payment System — Enterprise Product & UX Specification

Document Type: Product & UX Specification (Enterprise-Grade) **Source Document:** Vitaay Escrow Payment System PRD v1.0 (June 2026)
This Document Version: 2.0 **Status:** Draft for Cross-Functional Review **Prepared For:** Product, Engineering, UI/UX Design, Compliance, QA, Business Stakeholders **Classification:** Internal — Confidential

Version History

Version	Date	Author Role	Summary of Changes
1.0	June 2026	Product	Original PRD — escrow, tips, payouts, engineering status
1.1	2026-07-20	Engineering	Auto-release cron removed/descoped; releases now brand-triggered only
2.0	2026-08-04	Product/UX/Compliance (this doc)	Full expansion into enterprise spec: architecture, all payment states/edge cases, complete UI spec for 20 screens, compliance (PCI-DSS/GDPR/PSD2/RBI/AML/KYC), security model, error catalogue, notification matrix, API↔UI mapping, QA test plan, analytics, admin requirements, roadmap

Change control: Any modification to escrow state transitions, fee calculation, or compliance sections requires sign-off from Engineering Lead + Compliance Lead before merge.

Table of Contents

- 1. Executive Summary
- 2. Product Vision, Goals & KPIs
- 3. The Problem
- 4. Solution Overview
- 5. Users & Roles
- 6. Payment Architecture
- 7. Complete Payment Flows (Happy Path + Every Edge Case)
- 8. Escrow Status State Machine
- 9. UI/UX Specification (20 Screens)
- 10. Worldwide UX Benchmarking
- 11. Compliance
- 12. Security
- 13. UI Checklists
- 14. Error Catalogue
- 15. Notifications Matrix
- 16. API ↔ UI Mapping
- 17. QA Test Plan
- 18. Analytics
- 19. Admin Requirements

- 20. Engineering Status
- 21. Go-Live Readiness
- 22. Risks & Mitigations
- 23. Future Roadmap (Phase 2+)
- 24. Glossary
- 25. Appendices

1. Executive Summary

The Vitaay Payment System is a Stripe-powered escrow solution that makes payments between brands and creators safe, trustworthy, and automatic. It is the financial backbone of the Vitaay marketplace — without it there is no safe way for brands and creators to transact.

Core guarantee: - **Brands** only pay when they choose to release funds against a deliverable they’ve reviewed. - **Creators** get money that is verifiably held (not promised) from the moment a brand funds a campaign, removing the “will I actually get paid” risk that dominates informal creator-brand transactions.

What changed since v1.0 of the PRD: The auto-release-after-5-days mechanism described in the original design has been **removed** (2026-07-20). Escrow release is now a **brand-triggered action only** — there is no cron-based timeout release. This is a material change to the trust model (creators no longer have a system-guaranteed release window) and is treated as a first-class product risk in §22 and a Phase 2 candidate for reintroduction with safeguards.

What this document adds beyond the original PRD: a formal architecture description, a complete state machine, every failure/edge-case flow (not just the happy path), a full UI specification for all 20 screens with accessibility and responsive behavior, a compliance framework spanning PCI-DSS/GDPR/PSD2/SCA/AML/KYC/RBI/card-network rules, a security model, an exhaustive error catalogue, a notification matrix, an API↔UI traceability table, a QA test plan, an analytics event taxonomy, and admin/ops requirements.

2. Product Vision, Goals & KPIs

2.1 Vision Statement

Vitaay becomes the marketplace where **money is never the reason a brand-creator relationship breaks down**. Every campaign payment is protected by design, every creator payout is predictable, and every dispute has a resolution path that doesn’t require leaving the platform.

2.2 Business Goals

Goal	Description	Owner
Trust infrastructure	Make Vitaay the default safe way to pay in the creator economy, ahead of informal bank transfer/PayPal	Product
Revenue	Establish a durable transaction-fee revenue stream (8% platform fee)	Finance/Product
Liquidity/retention	Reduce creator churn caused by non-payment; reduce brand churn caused by fraud/no-delivery	Growth
Compliance moat	Be the only marketplace in this segment with full PCI-DSS/GDPR/PSD2/RBI compliance, turning compliance into a sales argument for enterprise brands	Compliance/Legal
Platform extensibility	Build the payment core so that milestones, subscriptions, and multi-currency can be layered on without a rewrite	Engineering

2.3 Success Criteria (Launch Definition of Done)

- Brand can fund, and release or refund, a campaign escrow end-to-end in production Stripe mode.

- Creator can complete Stripe Connect onboarding and receive a real bank payout.
- 100% of the 20 UI screens in §9 are implemented and pass their acceptance criteria.
- Zero P0/P1 items open in §21.
- Compliance sign-off obtained (Legal + Finance) on fee structure, ToS, and GST handling.

2.4 KPIs

See §18 Success Metrics table for the full KPI tree; summarized targets:

Metric	Target	Measured
Payment dispute rate	< 2%	Monthly
Creator Stripe onboarding completion	> 70% of invited creators	Monthly cohort
Brand payment conversion (page view → paid)	> 85%	Weekly
Median time to release payment	< 24 hours	Weekly
Platform fee revenue	£5,000+ by month 3	Monthly
Refund rate	< 5%	Monthly
Creator NPS on payment experience	> 60	Quarterly

3. The Problem

3.1 Without a Payment System

Pain Point	Who It Hurts	Impact
Brands pay creators directly (bank transfer, PayPal) with no protection	Brands	Creator doesn't deliver → brand loses money, no recourse
Creators complete work before receiving payment	Creators	Brand ghosts or disputes → creator worked for free
No standardised invoicing or fee structure	Both	Manual negotiations on payment method, delays, confusion
Platform has no revenue model	Vitaay	No sustainable business without transaction fees
Payment disputes are handled via email/messages	Both	Time-consuming, no resolution mechanism

3.2 Business Impact

- 15%+ dispute rate on informal payments in creator marketplaces (industry baseline).
- 30%+ creator churn attributable to late/missing payments.
- Brands lose trust in the platform, leading to reduced campaign spend.

3.3 Why This Is Urgent Now

Every week Vitaay operates without escrow, brand-creator deals happen off-platform (via direct bank transfer) to avoid fees, which both suppresses platform revenue and removes Vitaay from the transaction entirely, eliminating any dispute-resolution leverage.

4. Solution Overview

An escrow system means: *“the money is held by a neutral third party (Vitaay + Stripe) until both sides fulfil their obligations.”*

Brand pays → Money held safely by Stripe → Creator delivers → Brand approves → Creator gets paid

Note on the auto-release path: The original design included an automatic release after 5 days of brand inaction. This has been removed as of 2026-07-20. See §8 and §22 for the current model and the risk this introduces.

5. Users & Roles

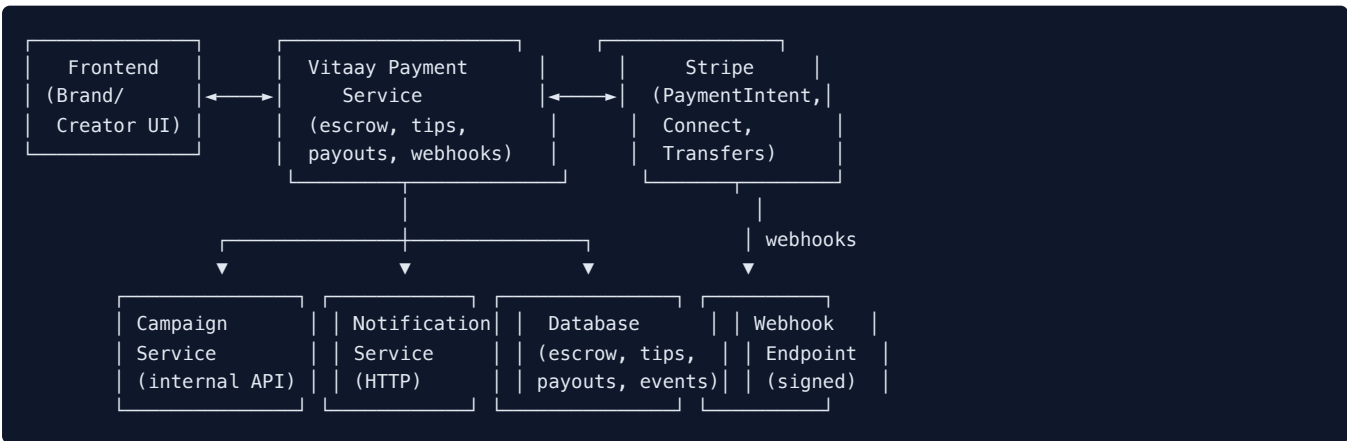
User	Role in Payments	What They See	Auth/Permission Notes
Brand	Pays for campaigns. Funds escrow. Approves deliverables to release payment. Can request refunds.	Payment form, escrow status, release/refund buttons	Must own the campaign/invite to act on its escrow (server-enforced)
Creator	Receives payments. Must complete Stripe onboarding once. Views earnings and payout history.	Stripe onboarding flow, earnings dashboard, payout history	Can only view own payouts; cannot self-release escrow
Admin	Monitors disputes. Can manually release or refund escrows.	Admin panel with escrow list, dispute queue	Elevated role, all actions audit-logged (see §19.2)
Any authenticated user	Can send a tip to a creator	Tip button on creator profile	Requires authentication; no special role needed

5.1 Role Capability Matrix

Action	Brand	Creator	Admin	Anonymous
Fund escrow	✓ (own campaign)	✗	✗	✗
Release escrow	✓ (own campaign)	✗	✓ (override)	✗
Refund escrow	✓ (own campaign, if Held)	✗	✓ (override)	✗
View escrow status	✓ (own)	✓ (own)	✓ (all)	✗
Send tip	✓	✓	✓	✗
Receive tip/payout	✗	✓	✗	✗
Complete Stripe onboarding	✗	✓	✗	✗
View audit logs	✗	✗	✓	✗

6. Payment Architecture

6.1 System Components



6.2 Stripe Building Blocks

Concept	Purpose in Vitaay
Stripe Connect (Express accounts)	Each creator gets a sub-account under Vitaay's Stripe platform account. Enables KYC-verified payouts without Vitaay handling bank data.
PaymentIntent	One PaymentIntent per escrow funding event; represents the brand's charge.
<code>transfer_data</code>	Used on the PaymentIntent to route funds toward the creator's Connect account at charge time, net of platform fee.
Direct charge / destination charge	Vitaay uses destination charges: brand's card is charged on the platform account, funds are held, then transferred to creator's Connect balance on release.
Transfers	Explicit transfer created on release, moving held funds from platform balance to creator Connect balance.
Refunds	Full refund issued against the original PaymentIntent/charge when escrow is refunded while Held.
Webhooks	Stripe → Vitaay event delivery for <code>payment_intent.succeeded</code> , <code>payment_intent.payment_failed</code> , <code>charge.refunded</code> , <code>transfer.created</code> , <code>account.updated</code> , <code>charge.dispute.created</code> , etc. (11 events handled — see §6.5).
Stripe Express Dashboard	Creator-facing, Stripe-hosted view of payout schedule and bank details. Opens in new tab; Vitaay has no access to raw bank data.

6.3 Escrow Lifecycle (Architectural View)

1. **Create** — Brand initiates funding. Backend creates a `PaymentIntent` with `transfer_data.destination` = creator's Connect account ID, amount = full campaign fee, `application_fee_amount` = platform's 8% cut computed server-side.
2. **Confirm** — Frontend (Stripe Elements) confirms the PaymentIntent client-side using `stripe.confirmPayment()`.
3. **Charge succeeds** — Stripe charges the brand's card, holds funds on the platform balance. Webhook `payment_intent.succeeded` updates escrow row to `Held`.
4. **Hold period** — No timer. Escrow stays `Held` indefinitely until brand acts.
5. **Release** — Brand calls release endpoint → backend creates a `Transfer` to the creator's Connect account → escrow → `Released`, payout record created.
6. **Refund** — Brand (or Admin) calls refund endpoint → backend issues a Stripe refund against the original charge → escrow → `Refunded`.

6.4 Internal Services & Data Flow

Service	Responsibility
payment-service	Owns escrow/tips/payouts tables, Stripe API calls, webhook ingestion, idempotency, fee calculation
campaign-service	Owns campaign/invite lifecycle; queries payment-service's internal API for escrow status by <code>campaignId</code> / <code>inviteId</code>
notification-service	Receives HTTP calls from payment-service on every payment event; fans out to email/push/SMS/in-app (see §15)
admin-service / admin panel	Reads escrow/dispute data; issues manual release/refund via payment-service's privileged endpoints

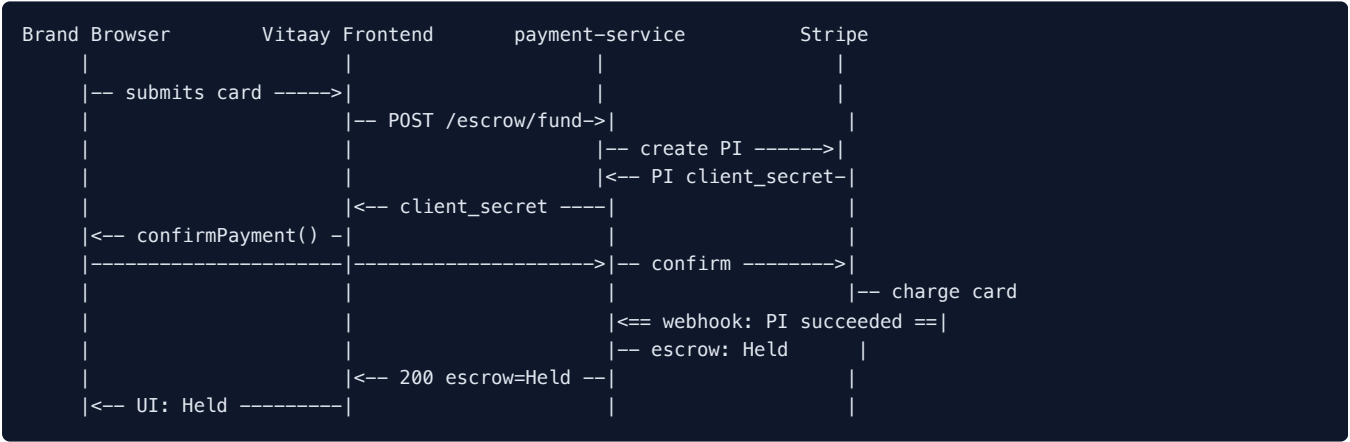
Notification integration is **HTTP-based**, not queue-based (no BullMQ) — this means a notification-service outage does not block escrow state transitions, but does risk silent notification loss; see §14 for the failure mode and §22.

6.5 Webhook Event Catalogue

Event	Effect
payment_intent.succeeded	Escrow → Held; notify creator
payment_intent.payment_failed	Escrow → Failed; notify brand
payment_intent.canceled	Escrow → Failed (cancelled variant)
charge.refunded	Escrow → Refunded (confirms refund settled); notify brand
transfer.created	Payout record created/confirmed for creator
transfer.failed	Payout marked Failed; notify creator to check bank details
payout.paid	Creator notified “landed in bank”
payout.failed	Creator notified; admin alerted
account.updated	Creator’s Connect onboarding status refreshed (Not Started → In Progress → Complete)
charge.dispute.created	Escrow → Disputed; admin alerted; brand/creator notified
charge.dispute.closed	Dispute resolved; escrow reconciled to Released/Refunded per outcome

All webhooks are **HMAC-signature verified** and **deduplicated** via a database-backed `webhook_events` table keyed on Stripe’s event ID (safe for multi-server/horizontally-scaled deployments — see §12.5).

6.6 Sequence Diagram — Fund → Hold



7. Complete Payment Flows (Happy Path + Every Edge Case)

7.1 Happy Path — Full Campaign Payment

```
Brand          Vitaay Platform          Stripe          Creator
|-- Fund Campaign ----->|
|          |-- Create PaymentIntent -->| |
|          |          |-- Charge card   |
|          |          |-- Hold funds    |
|          |<-- PI succeeded -----|
|<-- Escrow: HELD -----|
|    ... creator delivers content ...
|-- Approve & Release ----->|
|          |-- Transfer to Creator ---->| |
|          |          |-- Funds landed -->|
|<-- Escrow: RELEASED -----|          | Creator paid 🎉 |
```

7.2 Payment Failure (Card Declined at Funding)

```
Brand          Vitaay Platform          Stripe
|-- Fund Campaign -->|
|          |-- Create + confirm PI ---->| |
|          |          |-- Decline        |
|          |<-- PI failed (webhook) ----|
|<-- Escrow: FAILED --|
|<-- error: card_declined shown, "Try again" CTA --|
```

Escrow row created in `Pending` / `Failed`, never reaches `Held`. Brand may retry with a new `PaymentIntent` (new idempotency key).

7.3 Refund (Brand-Initiated, Escrow Held)

```
Brand          Vitaay Platform          Stripe
|-- Request Refund -->|
|          |-- Refund charge ----->|
|          |          |-- Return to card |
|<-- Escrow: REFUNDED--|
|    Money returns to brand's card (3-10 business days)
```

Guard: refund endpoint rejects the request server-side (403/409) if escrow status $\neq$ `Held`.

7.4 Chargeback / Dispute (Stripe-Initiated)

```
Cardholder Bank  Stripe          Vitaay Platform  Brand / Creator
|-- files dispute -->|
|          == webhook: dispute.created ==>|
|          |          |-- Escrow: DISPUTED |
|          |          |-- Admin alerted ----->|
|          |          |-- notify brand+creator|
|          |          (Stripe manages evidence submission externally)|
|          == webhook: dispute.closed ==>|
|          |          |-- Reconcile escrow   |
```

Vitaay does not build custom dispute-evidence UI in v1 — Stripe's dashboard/API handles evidence; Vitaay's job is to freeze escrow actions (no release/refund allowed) while `Disputed` and surface status to both parties.

7.5 Retry After Failure

Brand re-submits payment with a **new `PaymentIntent`** and a **new idempotency key** (never reuses the failed PI's key — Stripe would just return the same failure). Escrow record is updated in place (same escrow ID, new `stripe_payment_intent_id`) rather than creating a duplicate escrow row, to keep a 1:1 escrow↔campaign relationship.

7.6 Cancel (Brand Abandons Checkout Before Confirming)

No charge occurs; `PaymentIntent` stays `requires_payment_method / requires_confirmation` on Stripe's side and is never confirmed. Vitaay's escrow row (if created optimistically) stays `Pending` and is safe to re-enter later or expire per a data-retention job (not a financial state — no money moved).

7.7 Duplicate Payment Prevention (Double-Click / Double-Submit)

```
Brand Browser (double-click "Pay Now")
|-- POST /escrow/fund (Idempotency-Key: abc123) -->|
|-- POST /escrow/fund (Idempotency-Key: abc123) -->| (2nd click, same key)
                                                    |-- Stripe returns cached
                                                    result for 2nd call;
                                                    only ONE charge occurs
```

Idempotency key is derived client-side per checkout session and reused across retried submissions within that session (see §12.5).

7.8 Network Failure Mid-Payment (Client Loses Connection After Charge, Before Response)

```
Brand Browser      Vitaay Backend      Stripe
|-- fund request --->|-- confirm PI ----->|
| (connection drops)|                    |-- charge succeeds
|                    |<-- PI succeeded -----|
|                    |-- escrow: Held (persisted server-side regardless of client connectivity)
| [client reconnects, GET /escrow/:id/status] |
|<-- Held -----|
```

Server-side state is authoritative; client polls/re-fetches status on reconnect rather than trusting its own optimistic UI state.

7.9 Webhook Delay

UI shows a **“Processing...”** interim state (max recommended wait ~10–15s) if the synchronous `confirmPayment()` response arrives before the asynchronous webhook has updated the DB row. Frontend polls `GET /escrow/:id/status` with backoff until it observes `Held`, or shows a “we’ll notify you” fallback message past a timeout threshold (e.g., 30s) rather than blocking indefinitely.

7.10 Webhook Duplicate Delivery

Stripe guarantees at-least-once delivery — the same event ID can arrive twice. The `webhook_events` dedup table (keyed on Stripe event ID) short-circuits reprocessing: second delivery is acknowledged (200 OK) but produces no state change or duplicate notification.

7.11 Timeout (Stripe API Slow/Unresponsive)

Backend applies a bounded timeout on Stripe API calls; on timeout, the fund/release/refund request fails gracefully with a `502`-class error and **no partial state change** — either the Stripe call fully succeeded (confirmed via a follow-up status check before erroring to the client) or the local escrow state remains unchanged. Never leave escrow in an ambiguous state; reconcile via webhook if the Stripe-side call actually landed.

7.12 Session Expiration Mid-Checkout

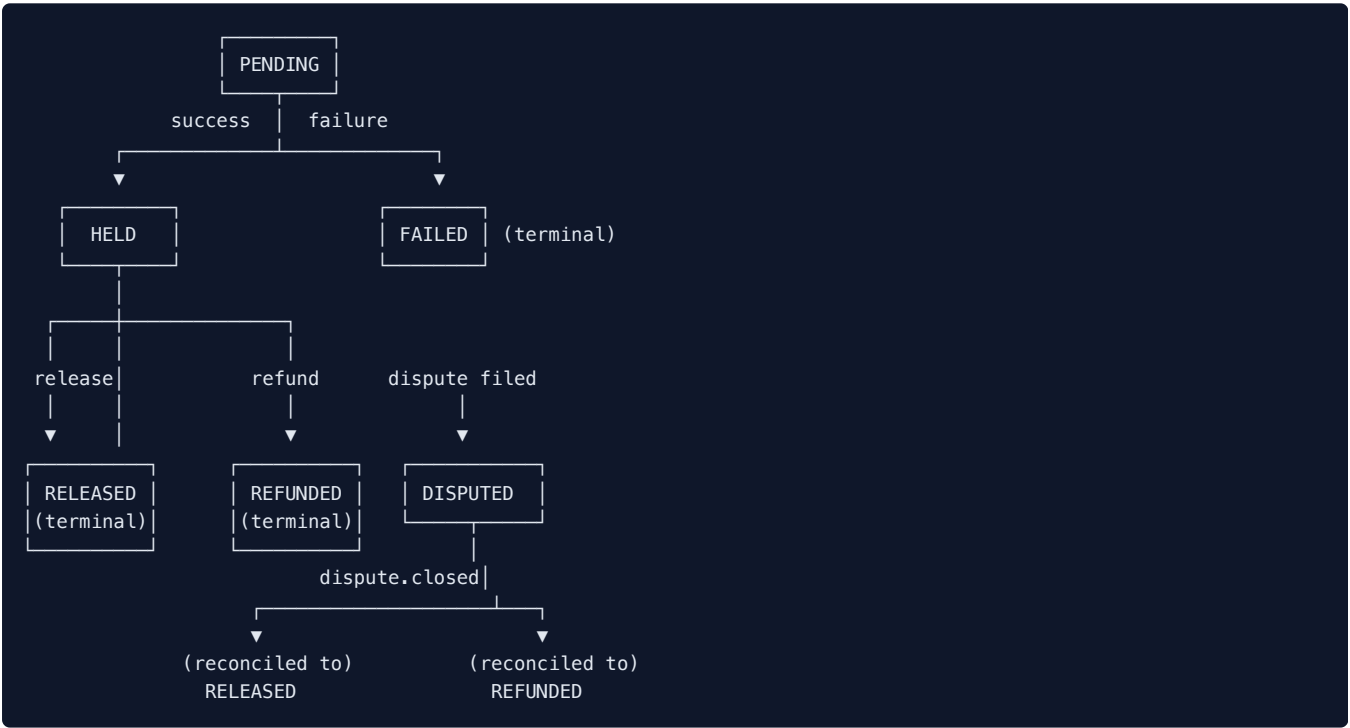
If the brand's auth session expires while on the funding screen, the `confirmPayment()` call (which talks to Stripe directly, not Vitaay) may still succeed, but the subsequent Vitaay-side status fetch will 401. UI must **re-authenticate and re-fetch escrow status** rather than assume payment failed — critical to avoid a brand believing they weren't charged when they were.

8. Escrow Status State Machine

8.1 States

Status	Meaning	Who Can Trigger	Next Possible Status
🟡 Pending	Payment intent created but not yet charged	System	Held, Failed
🟢 Held	Money charged and held securely	System (on successful charge)	Released, Refunded, Disputed
✅ Released	Money sent to creator	Brand (manual release only)	(terminal)
🔴 Refunded	Money returned to brand	Brand or Admin	(terminal)
⚠️ Disputed	Chargeback raised with Stripe	Stripe (external)	Released, Refunded (resolved externally via <code>dispute.closed</code>)
❌ Failed	Payment failed or cancelled	System	(terminal)

8.2 State Diagram



8.3 Guard Rules

- Release only allowed from `Held`.
- Refund only allowed from `Held`.
- No action allowed while `Disputed` except by Stripe's external resolution (webhook-driven).
- `Failed` and terminal states are immutable — no endpoint may transition out of them.
- **No timer-based transition exists anywhere in this state machine** (auto-release removed 2026-07-20) — every non-system transition requires an explicit brand or admin action.

9. UI/UX Specification (20 Screens)

Each screen below follows a consistent template: **Purpose** · **User** · **Components** · **Buttons/Fields** · **Validation** · **Loading/Empty/Error States** · **Permissions** · **Accessibility** · **Responsive Behavior** · **Notifications Triggered** · **Acceptance Criteria**.

9.1 Screen 1 — Creator Payment Onboarding

- **Purpose:** Get a creator through Stripe Connect KYC so they can receive money.
- **User:** Creator.
- **Components:** Status banner, “Set Up Payments” CTA, progress indicator, “Check Status” button, completion badge, “View Dashboard” link.
- **Buttons/Fields:** Set Up Payments (primary), Continue Setup (if in-progress), Check Status, View Stripe Dashboard (post-completion).
- **Validation:** N/A on Vitaay side — all KYC field validation happens on Stripe-hosted pages.
- **Loading:** Skeleton banner while onboarding status is fetched on page load.
- **Empty State:** “Not started” — banner reads “Payment Setup Required” with CTA.
- **Error State:** If Stripe redirect fails or status check errors, show “Couldn’t check your setup status — retry” with a retry button, not a dead end.
- **Permissions:** Creator-only; a brand or admin viewing a creator profile does not see this control.
- **Accessibility:** Banner uses `role="status"` for screen-reader announcement of state changes; CTA has descriptive `aria-label` (“Set up Stripe payments — opens in new tab”).
- **Responsive:** Banner stacks vertically below 480px; CTA remains full-width tap target (min 44px height) on mobile.
- **Animations:** Subtle checkmark animation on transition to “Complete” state.
- **Notifications Triggered:** “Stripe onboarding complete” push/email on `account.updated` webhook confirming `charges_enabled` + `payouts_enabled`.
- **Acceptance Criteria:**
 - ☐ Three distinct visual states render correctly (not started / in progress / complete).
 - ☐ “Check Status” triggers a live re-fetch, not a cached value.
 - ☐ Creator cannot access Fund/Release/Refund UI (n/a — creator never sees those) but is blocked from receiving payouts until `payouts_enabled=true`, enforced server-side, not just hidden in UI.

9.2 Screen 2 — Brand Checkout / Fund Escrow

- **Purpose:** Collect payment and fund the campaign escrow.
- **User:** Brand.
- **Components:** Fee breakdown table, Stripe `PaymentElement`, campaign summary card, terms checkbox, “Pay Now” button.
- **Buttons/Fields:** Card input (via `PaymentElement`), Pay Now (primary, disabled until Element reports `complete`), Cancel.
- **Validation:** Stripe Elements handles card validation inline (real-time, PCI-scope-free). “Pay Now” stays disabled until Elements reports a valid payment method.
- **Loading:** “Pay Now” enters a spinner+disabled state immediately on click; no double-submit possible (see §7.7).
- **Empty State:** N/A (form always has content once campaign context loads); show skeleton while campaign fee data loads.
- **Error State:** Inline Stripe Element error (e.g., “Your card was declined”) rendered directly under the card field; page-level banner for non-card errors (e.g., campaign already funded by someone else).
- **Permissions:** Only the brand who owns the campaign/invite may access this screen; deep-linking as another brand returns 403.
- **Accessibility:** Fee breakdown table has proper `<th>` / `<td>` semantics for screen readers; form errors are announced via `aria-live="polite"`.
- **Responsive:** Fee table collapses to a stacked key/value list under 600px; Stripe Element remains full-width.
- **Animations:** None required beyond Stripe’s own Element transitions; avoid custom motion that could mask a loading/error state.
- **Notifications Triggered:** “Escrow funded” → creator (on success). “Payment failed” → brand (on failure).
- **Acceptance Criteria:**
 - ☐ Fee breakdown always matches server-calculated amounts (client display never authoritative for charge amount — see §12.3).
 - ☐ Card errors map to the catalogue in §14 with correct user-facing copy.
 - ☐ Successful payment navigates to Escrow Status Card / Success state within 2s of webhook confirmation or shows Processing state per §7.9.

9.3 Screen 3 — Stripe Payment (Embedded Element State)

- **Purpose:** The live card-entry sub-component embedded in Checkout (Screen 2) and the Tip Modal (Screen 12).
- **User:** Brand, or any tipping user.
- **Components:** Stripe `PaymentElement` iframe, supported payment method icons, secure-badge microcopy (“Payments secured by Stripe”).

- **Fields:** Card number, expiry, CVC, postal code (or wallet options — Apple Pay/Google Pay if enabled).
- **Validation:** Fully delegated to Stripe.js; Vitaay never inspects raw field values.
- **Loading:** Element shows its own internal shimmer while mounting.
- **Error State:** Field-level inline errors from Stripe (e.g., “Your card number is incomplete”).
- **Permissions:** Any authenticated user in a payment context.
- **Accessibility:** Stripe Elements are WCAG 2.2 AA compliant out of the box; Vitaay must not wrap them in a way that breaks tab order or focus trapping.
- **Responsive:** Element auto-resizes to container width; container must not force a fixed width below 280px.
- **Acceptance Criteria:**
 - ☐ No raw card data ever touches Vitaay’s network requests (verified via network tab in QA — see §17.4).

9.4 Screen 4 — Processing Screen

- **Purpose:** Bridge the gap between `confirmPayment()` returning and the webhook confirming state server-side (see §7.9).
- **User:** Brand, tipping user.
- **Components:** Spinner, “Processing your payment...” message, reassurance microcopy (“Don’t close this tab”).
- **Loading:** This is the loading state; times out to a “still processing, we’ll notify you” message after ~30s.
- **Error State:** If status polling eventually returns `Failed`, transition to Failure Screen (9.6).
- **Accessibility:** `aria-live="assertive"` announcement on state resolution (success/failure), not during the wait itself (avoid spamming screen readers).
- **Responsive:** Centered modal/full-screen state on mobile; inline card on desktop.
- **Acceptance Criteria:**
 - ☐ Never leaves the user in an indefinite spinner with no escape — always resolves to success, failure, or an explicit “check back later” state within the timeout window.

9.5 Screen 5 — Success Screen

- **Purpose:** Confirm escrow funded / release / refund / tip success.
- **User:** Brand, creator, tipping user (context-dependent copy).
- **Components:** Success icon/animation, amount confirmation, next-step CTA (e.g., “View Campaign”, “Back to Profile”).
- **Loading:** N/A — terminal state.
- **Empty State:** N/A.
- **Error State:** N/A (this screen only renders on confirmed success).
- **Accessibility:** Success message announced via `aria-live`; icon has `aria-hidden="true"` with text equivalent present.
- **Responsive:** Single-column centered layout at all breakpoints.
- **Notifications Triggered:** Context-dependent (see §15).
- **Acceptance Criteria:**
 - ☐ Displays the actual server-confirmed amount, never a client-optimistic guess.

9.6 Screen 6 — Failure Screen

- **Purpose:** Clear, recoverable error state for any failed payment action.
- **User:** Brand, tipping user.
- **Components:** Error icon, human-readable message (mapped from §14), “Try Again” and “Use Different Card” CTAs, support link.
- **Error State:** This is the error state; must map every Stripe decline code to friendly copy — never show raw Stripe error strings to end users.
- **Permissions:** N/A.
- **Accessibility:** Error announced via `aria-live="assertive"` immediately on render (this is the one case where assertive is correct — it’s a blocking failure the user must act on).
- **Responsive:** CTAs stack full-width on mobile.
- **Notifications Triggered:** “Payment failed” → brand.
- **Acceptance Criteria:**
 - ☐ “Try Again” re-opens Checkout with a **fresh** `PaymentIntent`/idempotency key, never resubmits the failed one.

9.7 Screen 7 — Escrow Status Card

- **Purpose:** Persistent at-a-glance escrow state, embedded in the campaign detail page.
- **User:** Brand + Creator (both see it, different actions available).
- **Components:** Colour-coded status badge (🟡🟢🟠🔴⚠️❌), amount, funded date, released/refunded date if applicable, action buttons (context-sensitive).
- **Buttons:** `Release Payment` (brand, only when Held), `Request Refund` (brand, only when Held), none for creator (view-only).
- **Loading:** Skeleton badge while status loads.
- **Empty State:** “No payment yet” if campaign hasn’t been funded — shown to brand with a “Fund Campaign” CTA linking to Screen 2.
- **Error State:** “Couldn’t load payment status — retry” with retry button; never silently show a stale/wrong badge.
- **Permissions:** Release/Refund buttons rendered server-validated — even if shown, backend re-checks ownership + state on click.
- **Accessibility:** Status badge colour is never the only signal — always paired with text label (colour-blind safe).
- **Responsive:** Badge + buttons stack on mobile; buttons remain min 44px tap targets.
- **Acceptance Criteria:**
 - ☐ Badge state always matches the authoritative server state, refreshed on page load and after any action (no client-side-only optimistic status changes persisted).

9.8 Screen 8 — Release Payment Confirmation (Modal)

- **Purpose:** Prevent accidental release; confirm fee breakdown before an irreversible action.
- **User:** Brand.
- **Components:** Modal with “Release £460 to [Creator Name]?” headline, fee breakdown, `Confirm Release` / `Cancel` buttons.
- **Validation:** N/A (no input fields, confirmation only).
- **Loading:** `Confirm Release` shows spinner + disables both buttons on click.
- **Error State:** If release fails server-side (e.g., escrow no longer `Held` because a refund raced it), show inline modal error, do not close modal silently.
- **Permissions:** Same as Screen 7 — server re-validates on submit.
- **Accessibility:** Modal traps focus; `Escape` closes; initial focus on `Cancel` (safer default) not `Confirm Release`.
- **Responsive:** Full-screen modal on mobile, centered dialog on desktop.
- **Notifications Triggered:** “Payment released” → creator.
- **Acceptance Criteria:**
 - ☐ Irreversibility is stated explicitly in the modal copy (“This cannot be undone”).
 - ☐ Race condition (escrow state changed between modal open and confirm) surfaces a clear error, not a silent failure.

9.9 Screen 9 — Refund Confirmation (Modal)

- **Purpose:** Confirm refund amount and optionally capture a reason.
- **User:** Brand.
- **Components:** Modal “Refund £500?” headline, optional reason textarea/dropdown, `Confirm Refund` / `Cancel`.
- **Fields:** Reason (optional, for analytics — not required to submit).
- **Validation:** Reason field, if used, capped at a reasonable length (e.g., 500 chars) with a live counter.
- **Loading/Error:** Same pattern as Screen 8.
- **Permissions:** Only visible/actionable when escrow is `Held` (see §8.3).
- **Accessibility:** Same modal-focus rules as Screen 8.
- **Responsive:** Same as Screen 8.
- **Notifications Triggered:** “Refund issued” → brand.
- **Acceptance Criteria:**
 - ☐ Button is entirely absent (not just disabled) once escrow leaves `Held`, to avoid a confusing disabled-but-visible affordance.

9.10 Screen 10 — Transaction History

- **Purpose:** Full log of all payment-related events for a given campaign or account.
- **User:** Brand (their campaigns), Creator (their payouts — see 9.11 for the dedicated earnings variant), Admin (all).
- **Components:** Filterable/sortable table: date, type, amount, currency, status, related campaign link.
- **Fields:** Filter by type, date range, status; search by campaign name.
- **Loading:** Table skeleton rows.

- **Empty State:** “No transactions yet” illustration + copy appropriate to role.
- **Error State:** “Couldn’t load transactions” with retry.
- **Permissions:** Row-level scoping enforced server-side per role.
- **Accessibility:** Table sortable via keyboard; column headers are actual `<th scope="col">`.
- **Responsive:** Converts to card-per-row layout under 700px.
- **Acceptance Criteria:**
 - ☐ Pagination or infinite scroll handles 1000+ rows without perf degradation.

9.11 Screen 11 — Creator Earnings Dashboard

- **Purpose:** The creator’s “money page” — premium, trustworthy summary of all income.
- **User:** Creator.
- **Components:** Total earnings (lifetime + this month) summary cards, pending balance (in Stripe, not yet paid to bank), payout list (campaigns + tips combined), filters, “View in Stripe Dashboard” link.
- **Fields:** Filter by type (campaign/tip), date range.
- **Loading:** Skeleton summary cards + table.
- **Empty State:** “No earnings yet” with encouragement copy + link to profile completion / how payments work.
- **Error State:** Retry banner; summary cards show “—” rather than £0 if the fetch failed (avoid implying zero earnings incorrectly).
- **Permissions:** Creator sees only their own data.
- **Accessibility:** Summary numbers have `aria-label`s with full context (“Total lifetime earnings: four hundred sixty pounds”).
- **Responsive:** Summary cards stack 1-per-row on mobile, 2–3 per row on desktop.
- **Acceptance Criteria:**
 - ☐ Pending balance is visibly distinguished from “already paid out” balance (this is a common source of creator confusion in marketplace payment UX — see §10).

9.12 Screen 12 — Tip Button + Modal

- **Purpose:** Let any user send a direct tip to a creator from their public profile.
- **User:** Any authenticated user; entry point on the creator’s public profile.
- **Components:** “💰 Send Tip” button on profile, modal with amount input, preset amount chips (£5/£10/£25/£50), optional message field, `PaymentElement`, “Send Tip” button.
- **Fields:** Amount (custom or preset; min £1 / ₹100), optional message (capped length, e.g., 200 chars).
- **Validation:** Amount below minimum blocks submission with inline error; message length live-counted.
- **Loading:** Same pattern as Checkout (Screen 2).
- **Empty State:** N/A.
- **Error State:** Same card-decline handling as Checkout.
- **Permissions:** Requires authentication; unauthenticated visitors see a “Log in to send a tip” prompt instead of the modal.
- **Accessibility:** Preset chips are real buttons (not divs) with clear selected-state `aria-pressed`.
- **Responsive:** Modal is full-screen on mobile.
- **Notifications Triggered:** “Tip received” → creator (includes sender name + optional message).
- **Acceptance Criteria:**
 - ☐ Currency minimum is enforced server-side, not just client-side.

9.13 Screen 13 — Brand Dashboard (Payments Widget)

- **Purpose:** Summarize a brand’s active/past escrows within their broader campaign dashboard.
- **User:** Brand.
- **Components:** Widget listing active campaigns with escrow status badges, total spend summary, quick links to fund/release/refund actions.
- **Loading/Empty/Error:** Standard patterns as above.
- **Permissions:** Brand sees only own campaigns.
- **Accessibility/Responsive:** Standard patterns.
- **Acceptance Criteria:**
 - ☐ Widget status badges stay in sync with Screen 7 (single source of truth, no divergent caching).

9.14 Screen 14 — Notification Center

- **Purpose:** In-app feed of all payment-related notifications (funded, released, refunded, tip received, onboarding complete, payout landed, failures).
- **User:** All authenticated users.
- **Components:** List of notification cards, read/unread state, deep-link to relevant screen (e.g., tapping a “Payment released” notification opens Screen 7 for that campaign).
- **Loading/Empty (“You’re all caught up”)/Error:** Standard patterns.
- **Accessibility:** Unread indicator not colour-only (also a dot + “New” label).
- **Responsive:** Full list view on mobile; dropdown panel on desktop.
- **Acceptance Criteria:**
 - ☐ Every event in §15 with an in-app channel produces a corresponding Notification Center entry.

9.15 Screen 15 — Payment Settings

- **Purpose:** Creator’s hub for onboarding status, bank management link, payout schedule info.
- **User:** Creator.
- **Components:** Onboarding status summary (mirrors Screen 1), “Manage Bank Details” link (opens Stripe dashboard), payout schedule explainer (“Payouts land every Friday”).
- **Permissions:** Creator-only.
- **Accessibility/Responsive:** Standard patterns.
- **Acceptance Criteria:**
 - ☐ No banking data is ever rendered in Vitaay’s own UI — link-out only.

9.16 Screen 16 — Bank Settings (Stripe-Hosted, External)

- **Purpose:** Stripe Express Dashboard, opened in a new tab from Screens 1/15.
- **User:** Creator.
- **Vitaay Responsibility:** Only the link-out and returning-user redirect handling; no custom UI to design beyond a “You’re now leaving Vitaay” transition if legally/UX-required.
- **Acceptance Criteria:**
 - ☐ Opens in a new tab (`target="_blank"` with `rel="noopener noreferrer"`), preserving the Vitaay session in the original tab.

9.17 Screen 17 — Admin Escrow Management

- **Purpose:** List all escrows, filter by status, manually release/refund.
- **User:** Admin.
- **Components:** Filterable/sortable table (all escrows), status filter chips, search by brand/creator/campaign, row-level `Release / Refund` actions with the same confirmation-modal pattern as Screens 8/9 but labelled as an **admin override**.
- **Validation:** Admin override requires a mandatory reason field (unlike the optional one for brand-initiated refunds) — this is an audit requirement.
- **Permissions:** Admin role only, server-enforced.
- **Accessibility/Responsive:** Standard patterns; admin tools generally optimized for desktop but must remain usable on tablet.
- **Notifications Triggered:** Same as brand-initiated release/refund, plus an internal admin-action audit log entry.
- **Acceptance Criteria:**
 - ☐ Every admin override is logged with admin identity, timestamp, reason, and before/after state (see §19.2).

9.18 Screen 18 — Audit Logs

- **Purpose:** Immutable, searchable record of every sensitive payment action (admin overrides, webhook-driven state changes, refunds, releases).
- **User:** Admin.
- **Components:** Table with actor, action, target (escrow/tip/payout ID), before-state, after-state, timestamp, source (user action vs. webhook vs. system).
- **Fields:** Filter by actor, action type, date range, target ID.
- **Permissions:** Admin-only; logs themselves are append-only, no delete/edit UI exists.
- **Accessibility/Responsive:** Standard patterns.

- **Acceptance Criteria:**
 - ☐ Log entries are immutable at the database layer (no update/delete path), not just hidden in UI.

9.19 Screen 19 — Support Screen

- **Purpose:** Self-serve help + escalation path for payment issues (declined card, missing payout, dispute question).
- **User:** Brand, Creator.
- **Components:** FAQ accordion (common errors from §14 rewritten as help copy), “Contact Support” form pre-filled with relevant escrow/transaction ID when launched from a payment context.
- **Loading/Empty/Error:** Standard patterns.
- **Accessibility/Responsive:** Standard patterns.
- **Acceptance Criteria:**
 - ☐ Support form auto-attaches the relevant transaction/escrow ID and recent status history when deep-linked from a Failure Screen or Escrow Status Card, reducing back-and-forth.

9.20 Screen 20 — Dispute Center

- **Purpose:** Surface chargeback/dispute status to brand and creator (read-only in v1 — evidence submission stays in Stripe).
- **User:** Brand, Creator, Admin.
- **Components:** Dispute status card (mirrors Escrow Status Card styling but with ⚠ Disputed state), explainer copy (“This payment has been disputed by the cardholder’s bank. Vitaay is reviewing it with Stripe.”), no action buttons for brand/creator (frozen state), Admin sees a link to the Stripe dashboard dispute record.
- **Permissions:** Read-only for brand/creator; admin has an external link only (no in-app evidence tooling in v1 — Phase 2 candidate, see §23).
- **Accessibility/Responsive:** Standard patterns.
- **Notifications Triggered:** “Payment disputed” → brand + creator (add this to the notification matrix as a v2.0 addition — flagged, since the original PRD’s notification table did not include a dispute-created event; see §15).
- **Acceptance Criteria:**
 - ☐ Release/Refund actions are impossible to trigger anywhere in the product while an escrow is Disputed (defense in depth — UI hides them, API rejects them).

10. Worldwide UX Benchmarking

Pattern	Stripe	PayPal	Adyen	Amazon	Airbnb	Fiverr	Upwork	Shopify
Fee transparency	Line-itemized before charge	Often bundled, less transparent	Line-itemized (merchant-configurable)	Bundled into total	Itemized service fee shown pre-book	Itemized “Fiverr fee”	Itemized service fee	Itemized checkout
Escrow-style holding language	N/A (not escrow)	“Payment on Hold” for buyer protection	N/A	N/A	Host payout held until 24h after check-in	“Clearance period” before funds available	Funds held until milestone approval, with a scheduled auto-release	N/A

Pattern	Stripe	PayPal	Adyen	Amazon	Airbnb	Fiverr	Upwork	Shopify
Pending vs. available balance distinction	Balance vs. Payouts tabs	"Available"/"Pending" clearly split	Merchant-configurable	N/A	Host "Upcoming payouts" vs "Paid out"	Clearance period countdown shown per-order	"Available"/"Pending"/"In escrow"	Balance vs. Payor
Confirmation before irreversible action	Confirmation dialogs on destructive dashboard actions	Confirmation on refund/dispute response	Merchant-configurable	Order cancellation confirmation	Cancellation policy shown before confirming	Order cancellation confirmation	Contract end confirmation	Refund confirmation modal
Dispute self-service	Full dashboard evidence flow	In-app resolution center	Merchant dashboard	Amazon A-to-z Guarantee flow	Resolution Center	Resolution Center	Dispute flow with mediation	Limited (r to Shop Pay/proc
Trust badges/security microcopy	"Powered by Stripe" ubiquitous	PayPal Buyer Protection badge	N/A (invisible infra)	"Secure transaction"	N/A	"Secure payments powered by X"	Escrow explainer during first-time payment	"Secure checkout

Key takeaway: Vitaay's overall design direction (fee transparency, confirmation modals, status badges) already matches best-in-class marketplace UX. The two gaps versus category leaders (Upwork especially) are: (1) no visible "when will this release" expectation-setting now that auto-release is removed, and (2) no in-app dispute resolution center. Both are addressed as explicit risks/roadmap items in §22/§23.

11. Compliance

For every item: **Requirement** · **Reason** · **Backend Responsibility** · **Frontend Responsibility** · **UX Requirement** · **Risk if Missing** · **Priority**.

11.1 PCI-DSS

Item	Detail
Requirement	Card data never touches Vitaay's servers or is logged anywhere in Vitaay's stack
Reason	PCI-DSS SAQ A eligibility requires full outsourcing of card handling to a compliant processor (Stripe)
Backend Responsibility	Never accept raw PAN/CVC in any request body; only accept Stripe tokens/PaymentMethod IDs
Frontend Responsibility	Use Stripe Elements/ <code>PaymentElement</code> exclusively for card input; never build custom card fields
UX Requirement	"Payments secured by Stripe" trust microcopy near every card field

Item	Detail
Risk if Missing	Loss of PCI compliance, potential card-network fines, breach liability
Priority	P0

11.2 GDPR (and UK GDPR)

Item	Detail
Requirement	Lawful basis for processing brand/creator payment data; data minimization; right to access/erasure; data processing agreement with Stripe
Reason	Vitaay operates with GBP-denominated flows and UK/EU users
Backend Responsibility	Store only the minimum payment metadata needed (amounts, statuses, Stripe IDs) — never mirror KYC documents; support data export/erasure requests (noting financial records may have statutory retention overriding erasure)
Frontend Responsibility	Clear consent/notice at onboarding and checkout referencing the privacy policy; no dark patterns in tip/checkout flows
UX Requirement	Privacy policy link visible at first payment touchpoint (onboarding, checkout, tip modal)
Risk if Missing	Regulatory fines up to 4% global turnover; reputational damage
Priority	P0

11.3 PSD2 / SCA (Strong Customer Authentication)

Item	Detail
Requirement	Payments involving EEA-issued cards require two-factor authentication (3D Secure) unless exempted
Reason	Regulatory requirement for EEA card transactions
Backend Responsibility	Use Stripe PaymentIntents (already SCA-ready by design) and correctly handle <code>requires_action</code> status
Frontend Responsibility	Handle the 3DS challenge modal Stripe.js triggers automatically; never bypass or suppress it
UX Requirement	Processing Screen (9.4) must gracefully accommodate the extra 3DS step without appearing “stuck”
Risk if Missing	Declined transactions, non-compliance penalties from acquiring bank
Priority	P0 (for any EEA brand transactions)

11.4 3D Secure

Item	Detail
Requirement	Support the 3DS challenge flow end-to-end
Reason	Component of SCA/PSD2; also reduces fraud liability shift to card issuer
Backend Responsibility	Correctly interpret <code>requires_action</code> PaymentIntent status and relay to frontend
Frontend Responsibility	Call <code>stripe.confirmPayment()</code> in a way that supports Stripe's automatic 3DS modal
UX Requirement	Do not assume synchronous confirm/fail — architecture must accommodate the extra round trip already covered by 9.4/9.9
Risk if Missing	Fraud liability falls on Vitaay/brand instead of issuer; declined payments

Item	Detail
Priority	P0

11.5 AML (Anti-Money Laundering)

Item	Detail
Requirement	Monitor for structuring/suspicious payment patterns (e.g., rapid tip cycling, unusually large campaign payments)
Reason	Marketplace payment flows are an AML vector if unmonitored
Backend Responsibility	Rely on Stripe's built-in Radar/AML tooling as first line; define internal velocity thresholds that flag to Admin (see §12.9)
Frontend Responsibility	None directly; admin tooling surfaces flags (Screen 17/Admin)
UX Requirement	N/A for end users; Admin sees a "Flagged for Review" state
Risk if Missing	Regulatory exposure, platform used for laundering
Priority	P1

11.6 KYC (Know Your Customer)

Item	Detail
Requirement	Creators verified before receiving payouts
Reason	Regulatory requirement for payout-receiving accounts
Backend Responsibility	Gate payout/transfer creation on <code>payouts_enabled=true</code> from Stripe Connect account status
Frontend Responsibility	Screen 1/15 clearly block payment receipt until onboarding complete
UX Requirement	No campaign invite should let a brand fund an escrow destined for a creator who hasn't completed KYC — flag or block at invite/fund time
Risk if Missing	Regulatory violation, payouts stuck in limbo, poor creator experience
Priority	P0

11.7 OFAC / Sanctions Screening

Item	Detail
Requirement	No payments to sanctioned individuals/entities
Reason	Legal requirement, delegated largely to Stripe's onboarding checks
Backend Responsibility	Trust Stripe Connect's built-in OFAC screening during KYC; do not attempt to bypass or pre-approve accounts outside Stripe's verification
Frontend Responsibility	None
UX Requirement	If Stripe rejects an account for sanctions reasons, show a generic "unable to complete setup" message — never expose the specific reason to avoid legal/PR issues
Risk if Missing	Severe legal/regulatory penalties
Priority	P0

11.8 RBI Guidelines (India)

Item	Detail
Requirement	Compliance with RBI's payment aggregator/intermediary rules for INR transactions; GST line-item handling
Reason	PRD explicitly flags India-specific tips (₹100 minimum) and pending GST review
Backend Responsibility	Ensure INR flows route through RBI-compliant rails (Stripe's India entity or equivalent licensed PA); compute GST as a distinct, auditable line item once legal confirms structure
Frontend Responsibility	Display GST breakdown for Indian transactions once implemented (currently a gap — see §21)
UX Requirement	Currency-appropriate minimums and fee breakdowns (₹100 tip minimum already specified)
Risk if Missing	Illegal payment aggregation in India, transaction blocking, fines
Priority	P0 (blocking for India go-live specifically)

11.9 Visa / Mastercard Network Rules

Item	Detail
Requirement	Correct merchant category coding, dispute response timelines, refund policy disclosure
Reason	Card network compliance is a precondition for continued card acceptance
Backend Responsibility	Ensure Stripe account is registered under an accurate MCC for marketplace/creator-economy payments
Frontend Responsibility	Refund/cancellation policy must be discoverable pre-payment (linked from Checkout, Screen 2)
UX Requirement	Terms/refund policy link visible before “Pay Now”
Risk if Missing	Increased chargeback risk, potential account termination by acquirer
Priority	P1

11.10 Chargeback Guidelines & Refund Policies

Item	Detail
Requirement	Documented, consistent refund policy; timely chargeback response via Stripe
Reason	Reduces dispute rate (target < 2%, see KPIs) and protects merchant standing
Backend Responsibility	Freeze escrow actions during Disputed (already modeled in §8); log all evidence-relevant data (delivery timestamps, brand approval actions) for potential dispute evidence
Frontend Responsibility	Refund Confirmation modal (9.9) states policy clearly
UX Requirement	Refund policy consistent across ToS, Checkout, and Support screens
Risk if Missing	Higher dispute rate, lost chargebacks, network penalties
Priority	P1

11.11 Consumer Protection

Item	Detail
Requirement	Clear pre-payment disclosure of total cost, fee structure, and cancellation rights

Item	Detail
Reason	UK/EU consumer protection law (and general best practice)
Backend Responsibility	Server-calculated, tamper-proof fee display (see §12.3)
Frontend Responsibility	Fee breakdown always shown before any charge (Checkout, Tip Modal)
UX Requirement	No surprise fees post-charge
Risk if Missing	Regulatory complaint, consumer trust erosion
Priority	P0

11.12 Accessibility (WCAG 2.2 AA)

Item	Detail
Requirement	All 20 payment screens meet WCAG 2.2 AA
Reason	Legal requirement in many jurisdictions for commercial services; also simply good practice for a payments product
Backend Responsibility	N/A directly, but error messages/API responses should carry machine-readable codes so frontend can render accessible copy (see §14)
Frontend Responsibility	Colour-independent status signaling, proper focus management in modals, <code>aria-live</code> regions, keyboard navigability, 44px minimum tap targets, 4.5:1 contrast minimum
UX Requirement	Baked into every screen spec in §9
Risk if Missing	Legal exposure, excludes disabled users from a financial product
Priority	P0

12. Security

12.1 Card Security & Tokenization

Card data is tokenized entirely client-side by Stripe.js before it ever leaves the browser. Vitaay's backend only ever sees a `PaymentMethod` ID or `PaymentIntent` client secret — never a PAN, CVC, or full card number.

12.2 Idempotency

Every state-changing Stripe operation (fund, release, refund) is called with an idempotency key generated once per logical user action and reused on retries, preventing duplicate charges/transfers/refunds on network retry or double-click.

12.3 Server-Side Fee Calculation

The 8% platform fee is always calculated on the backend at the moment of charge creation. Any fee value submitted by the client is ignored — the server is the sole source of truth for `application_fee_amount`.

12.4 Webhook Signature Verification

Every incoming Stripe webhook is verified via HMAC signature (`Stripe-Signature` header) against the endpoint's signing secret before any processing occurs. Unverified/invalid-signature requests are rejected with 400 and never touch business logic.

12.5 Idempotency & Replay Protection

Webhook events are deduplicated against a database-backed `webhook_events` table keyed on Stripe's event ID — safe under horizontal scaling/multi-server deployment, and immune to replay attacks (an attacker resending a captured, validly-signed-but-already-processed event produces no effect).

12.6 Encryption

Data in transit: TLS 1.2+ enforced on all API traffic. Data at rest: standard database-level encryption for stored payment metadata (escrow/tip/payout records); no card data is ever stored, so PCI scope for data-at-rest is minimal by design.

12.7 Rate Limiting

Payment-initiating endpoints (`/escrow/fund` , `/tips/send`) are rate-limited per user/IP to blunt card-testing/enumeration attacks; release/refund endpoints are similarly rate-limited given their financial impact.

12.8 CSRF / XSS / Clickjacking

- CSRF: state-changing payment endpoints require a valid session + anti-CSRF token (or equivalent same-site cookie protections) — never accept a GET request that changes escrow state.
- XSS: all user-supplied text (tip messages, refund reason) is sanitized/escaped before storage and rendering; Content-Security-Policy restricts inline scripts.
- Clickjacking: payment pages send `X-Frame-Options: DENY` / `frame-ancestors 'none'` — Stripe Elements are already clickjacking-resistant, but Vitaay's own checkout page must not be embeddable in a third-party iframe.

12.9 Fraud Detection & Velocity Checks

Leverage Stripe Radar for card-level fraud scoring. Layer Vitaay-specific velocity checks (e.g., N tips from one account within M minutes, unusually large campaign fundings from a new brand account) that flag to the Admin dispute/fraud queue rather than auto-blocking, to avoid false-positive friction on legitimate large brand spend.

12.10 Device Fingerprinting & Suspicious Activity

Where privacy regulations allow, incorporate Stripe Radar's device/session signals for fraud scoring rather than building custom fingerprinting — avoids duplicating a solved problem and keeps Vitaay out of additional data-handling obligations under GDPR.

12.11 Logging, Audit Trail, Secrets, Key Rotation, Session Security

Control	Requirement
Logging	Log all payment state transitions with actor, timestamp, before/after state; never log raw card data or full Stripe secret keys
Audit Trail	Immutable audit log for admin overrides (see §19.2)
Secrets	Stripe secret keys and webhook signing secrets stored in a secrets manager, never in source control or client-accessible config
Key Rotation	Documented rotation procedure for Stripe API keys and webhook secrets, at minimum on suspected compromise, ideally on a scheduled cadence
Session Security	Payment-initiating actions require a valid, non-expired session; session expiration mid-checkout is handled per §7.12, not silently ignored

13. UI Checklists

13.1 Checkout Checklist

- ☐ Fee breakdown matches server calculation exactly
- ☐ “Pay Now” disabled until Stripe Element reports complete
- ☐ No double-submit possible
- ☐ Card errors mapped to friendly copy (§14)
- ☐ Refund/terms policy linked pre-payment
- ☐ Works with keyboard only
- ☐ 3DS challenge does not break layout

13.2 Success Checklist

- ☐ Shows server-confirmed amount only
- ☐ Correct next-step CTA per context (fund/release/refund/tip)
- ☐ Screen-reader announces success

13.3 Failure Checklist

- ☐ Never shows raw Stripe error strings
- ☐ “Try Again” always uses a fresh idempotency key
- ☐ Support link present

13.4 Refund Checklist

- ☐ Button only visible when escrow is `held`
- ☐ Confirmation modal shows exact amount
- ☐ Reason field optional, capped length

13.5 Escrow Checklist

- ☐ Status badge never colour-only
- ☐ Status always server-fetched, not cached stale
- ☐ Release/Refund buttons re-validated server-side on click

13.6 Creator Dashboard Checklist

- ☐ Pending vs. paid-out balance visually distinct
- ☐ Filter by campaign/tip works
- ☐ “View in Stripe” opens correctly in new tab

13.7 Brand Dashboard Checklist

- ☐ Status badges match Escrow Status Card exactly (single source of truth)

13.8 Admin Checklist

- ☐ Every override requires a mandatory reason
- ☐ Every override is audit-logged with actor identity

13.9 Settings Checklist

- ☐ No banking data rendered anywhere in Vitaay UI
- ☐ Onboarding status always live, not cached across sessions

13.10 Accessibility Checklist

- ☐ 4.5:1 minimum contrast on all status badges/text
- ☐ 44px minimum tap targets
- ☐ Modals trap focus and support Escape
- ☐ All interactive elements reachable via keyboard alone
- ☐ `aria-live` used correctly (polite for status, assertive for blocking errors)

13.11 Mobile / Tablet / Desktop Checklist

- ☐ All 20 screens tested at 360px, 768px, 1024px, 1440px breakpoints
- ☐ Tables convert to card layout below 700px where specified
- ☐ Modals go full-screen on mobile

13.12 Dark Mode Checklist

- ☐ Status badge colours retain sufficient contrast in dark mode
- ☐ Stripe Elements dark-mode theme matches Vitaay's dark palette

13.13 Localization / Currency / Timezone / Language Checklist

- ☐ Currency symbol and formatting correct per locale (£, ₹, etc.)
- ☐ Dates rendered in user's local timezone, with absolute date available on hover/tap
- ☐ All payment copy externalized for translation (no hardcoded English strings in components)

14. Error Catalogue

Error	User Message	Developer Message	Retry?	Recovery	Logging	Monitoring
Card declined	"Your card was declined. Please try a different payment method."	<code>card_declined</code> from Stripe	Yes, new PaymentIntent	"Use Different Card" CTA	Log decline code (not card data)	Alert if decline rate spikes platform-wide
Insufficient funds	"Your card doesn't have enough funds for this payment."	<code>insufficient_funds</code>	Yes	Same as above	Log decline code	Standard
Expired card	"This card has expired. Please use a different card."	<code>expired_card</code>	Yes	Prompt new card entry	Log decline code	Standard
Incorrect CVC	"The security code you entered is incorrect."	<code>incorrect_cvc</code>	Yes	Inline retry, same card	Log decline code	Standard
Authentication failed (3DS)	"We couldn't verify this payment with your bank. Please try again."	<code>authentication_required</code> / 3DS failure	Yes	Re-trigger 3DS or new card	Log event	Standard
Stripe unavailable	"Payments are temporarily unavailable. Please try again shortly."	Stripe API 5xx/timeout	Yes, after delay	Auto-retry with backoff or manual retry	Log full error server-side	Page/alert on-call if sustained
Webhook timeout	(invisible to user — shows Processing state)	Webhook not received within expected window	N/A (system)	Poll status endpoint (§7.9)	Log missing webhook	Alert if webhook lag exceeds threshold
Duplicate payment attempt	(invisible — idempotency prevents double charge)	Duplicate idempotency key detected	N/A	Return cached original result	Log dedup hit	Standard
Refund failure	"We couldn't process this refund. Our team has been notified."	Stripe refund API error	Admin-assisted	Escalate to Admin/Support	Log full error	Alert on-call

Error	User Message	Developer Message	Retry?	Recovery	Logging	Monitoring
Transfer failure	"We couldn't release this payment. Our team has been notified."	Stripe transfer API error	Admin-assisted	Escalate to Admin/Support	Log full error	Alert on-call
Bank rejected payout	"Your bank rejected a recent payout. Please check your bank details."	<code>payout.failed</code> webhook	Creator updates bank details via Stripe dashboard	Notify creator with clear next step	Log event	Alert if payout failure rate spikes
Escrow state conflict (e.g., release attempted on non-Held escrow)	"This payment can no longer be released — its status has changed."	409 state-conflict error	No — user must refresh	Refresh Escrow Status Card	Log conflict attempt	Standard
Unauthorized action	"You don't have permission to do this."	403	No	Redirect appropriately	Log attempt (potential probing)	Alert on repeated 403s from same user

15. Notifications Matrix

Event	Channel(s)	Audience	Trigger	Priority	Retry
Escrow funded	Email, Push, In-App	Creator	<code>payment_intent.succeeded</code>	High	Retry HTTP call to notification-service up to N times with backoff
Payment released	Email, Push, In-App	Creator	Release endpoint success	High	Same
Auto release triggered	—	—	✗ Removed 2026-07-20 — no auto-release notification	—	—
Refund issued	Email, In-App	Brand	Refund endpoint success / <code>charge.refunded</code>	High	Same
Tip received	Push, In-App	Creator	Tip charge success	Medium	Same
Payment failed	Email, In-App	Brand	<code>payment_intent.payment_failed</code>	High	Same
Stripe onboarding complete	Email, In-App	Creator	<code>account.updated</code> (payouts_enabled true)	Medium	Same
Payout landed in bank	Push, In-App	Creator	<code>payout.paid</code>	Medium	Same
Payment disputed (new — added in this spec)	Email, In-App	Brand + Creator	<code>charge.dispute.created</code>	High	Same
Payout failed (new — added in this spec)	Email, In-App	Creator	<code>payout.failed</code> / <code>transfer.failed</code>	High	Same
Admin manual override executed (new — added in this spec)	In-App (Admin only)	Admin team	Manual release/refund via Screen 17	Medium	N/A (internal)

Gap identified vs. original PRD: the source document's notification table did not cover dispute-created, payout-failed, or admin-override events. These are added here as they are necessary for both parties to understand a Disputed / Failed state without

contacting Support blind.

16. API ↔ UI Mapping

API Endpoint	Frontend Screen	Button	Request	Response	Loading	Error	Success	Permissions
POST /connect/onboarding-link	Screen 1	Set Up Payments	{ }	{ url }	Spinner on button	Retry banner	Redirect to Stripe	Creator
GET /connect/status	Screen 1, 15	Check Status	—	{ status }	Skeleton	Retry banner	Updated badge	Creator
POST /escrow/fund	Screen 2	Pay Now	{ campaignId, paymentMethodId, idempotencyKey }	{ escrowId, status }	Processing screen (9.4)	Failure screen (9.6)	Success screen (9.5)	Brand (owner)
GET /escrow/:id/status	Screen 7, 4	(polling)	—	{ status, amount, dates }	Skeleton badge	Retry banner	Badge update	Brand + Creator (owner)
POST /escrow/:id/release	Screen 8	Confirm Release	{ escrowId }	{ status: 'released' }	Spinner in modal	Inline modal error	Success screen	Brand (owner)
POST /escrow/:id/refund	Screen 9	Confirm Refund	{ escrowId, reason? }	{ status: 'refunded' }	Spinner in modal	Inline modal error	Success screen	Brand (owner)
POST /tips/send	Screen 12	Send Tip	{ creatorId, amount, message?, paymentMethodId }	{ tipId, status }	Processing screen	Failure screen	Success screen	Any authenticated user
GET /creator/earnings	Screen 11	—	—	{ totals, pending, list }	Skeleton	Retry banner	Populated dashboard	Creator (owner)
GET /transactions	Screen 10, 13	—	filters	{ list, pagination }	Skeleton rows	Retry banner	Populated table	Role-scoped user
GET /notifications	Screen 14	—	—	{ list }	Skeleton	Retry banner	Populated feed	Any authenticated user
GET /admin/escrows	Screen 17	—	filters	{ list }	Skeleton	Retry banner	Populated table	Admin
POST /admin/escrows/:id/release	Screen 17	Release (override)	{ escrowId, reason } (reason mandatory)	{ status }	Spinner in modal	Inline modal error	Success + audit log entry	Admin
POST /admin/escrows/:id/refund	Screen 17	Refund (override)	{ escrowId, reason } (reason mandatory)	{ status }	Spinner in modal	Inline modal error	Success + audit log entry	Admin
GET /admin/audit-logs	Screen 18	—	filters	{ list }	Skeleton	Retry banner	Populated table	Admin
POST /webhooks/stripe	(server-only, no UI)	—	Stripe event payload	200/400	N/A	N/A	State transitions per §6.5	System (signature-verified)

17. QA Test Plan

17.1 Positive Scenarios

- Fund escrow with valid test card → status `Held`.
- Release a `Held` escrow → status `Released`, creator payout record created.
- Refund a `Held` escrow → status `Refunded`, brand's card credited.
- Send a tip → creator balance reflects tip minus platform fee.
- Complete Stripe onboarding in test mode → creator can subsequently receive a payout.

17.2 Negative Scenarios

- Attempt release on a `Refunded` or `Released` escrow → 409, no state change.
- Attempt refund on a `Released` escrow → 409, blocked.
- Attempt fund/release/refund as a non-owning brand → 403.
- Submit tip below minimum amount → validation error, no charge attempted.
- Submit card that triggers each Stripe test decline code in §14 → correct friendly message shown for each.

17.3 Boundary Scenarios

- Tip at exactly the minimum amount (£1 / ₹100) → succeeds.
- Tip at £0.99 / ₹99 → blocked client-side and server-side.
- Very large campaign amount (test Stripe's max supported amount) → succeeds or fails gracefully with a clear message, not a silent crash.
- Refund reason field at exactly the character cap → accepted; one character over → blocked/truncated with a clear counter.

17.4 Security Test Scenarios

- Inspect network tab during checkout to confirm no raw card data appears in any Vitaay-bound request.
- Attempt to replay a captured, validly-signed webhook payload → confirm it produces no duplicate state change (dedup working).
- Attempt to submit a client-supplied fee amount different from server calculation → confirm server ignores it.
- Attempt CSRF-style cross-site POST to `/escrow/:id/release` without a valid session/token → confirm rejection.
- Attempt to access another brand's escrow status via direct ID manipulation → confirm 403.

17.5 Accessibility Scenarios

- Complete full Checkout flow using keyboard only.
- Verify screen reader correctly announces Processing → Success/Failure transitions.
- Verify all status badges pass a colour-blindness simulation check (never colour-only).

17.6 Localization Scenarios

- Fund/tip in GBP and INR, confirm correct symbol, formatting, and minimums per currency.
- Confirm dates render in the viewing user's local timezone.

17.7 Performance / Load Scenarios

- Load Transaction History / Admin Escrow list with 1,000+ rows — confirm acceptable render time and working pagination.
- Simulate webhook burst (e.g., 100 events in a short window) — confirm dedup and processing keep up without dropped events.

17.8 Network Interruption / Retry / Duplicate Click / Session Expiry / Refresh Scenarios

- Double-click "Pay Now" rapidly → confirm only one charge occurs (idempotency).
- Kill network connection immediately after clicking "Pay Now," then restore → confirm UI reconciles to the true server state on reconnect rather than showing a false failure.
- Let session expire mid-Checkout, then attempt to confirm payment → confirm graceful re-auth + status re-fetch, not a false "not charged" message (per §7.12).

- Refresh the browser mid-“Processing” state → confirm the Escrow Status Card reflects the true current state on reload, not a reset “Pending” view.

18. Analytics

18.1 Core Events

Event	Properties
payment_started	context (fund/tip/release/refund), campaignId?, amount, currency, userRole
payment_success	context, escrowId / tipId, amount, currency, feeAmount, timeToConfirmMs
payment_failed	context, errorCode, amount, currency
refund_requested	escrowId, amount, reasonProvided (bool)
refund_completed	escrowId, amount
escrow_released	escrowId, amount, timeSinceFundedMs
creator_paid	payoutId, amount, type (campaign/tip)
dashboard_viewed	dashboardType (earnings/brand/admin), userRole
receipt_exported	transactionId, format
onboarding_started	creatorId
onboarding_completed	creatorId, timeToCompleteMs
dispute_created	escrowId, amount

18.2 Funnel Tracking

- **Brand checkout funnel:** payment_started → 3DS challenge shown (if applicable) → payment_success / payment_failed . Used to compute the “brand payment conversion rate” KPI (§2.4).
- **Creator onboarding funnel:** onboarding_started → Stripe redirect → onboarding_completed . Used to compute onboarding completion rate KPI.
- **Release latency funnel:** escrow_released.timeSinceFundedMs aggregated to compute “average time to release payment” KPI.

18.3 North Star & Supporting KPIs

Metric	Formula (from events)	Target
Brand payment conversion rate	payment_success / payment_started (context=fund)	> 85%
Onboarding completion rate	onboarding_completed / onboarding_started	> 70%
Median release latency	median(escrow_released.timeSinceFundedMs)	< 24h
Dispute rate	dispute_created / payment_success (context=fund)	< 2%
Refund rate	refund_completed / payment_success (context=fund)	< 5%

19. Admin Requirements

19.1 Admin Dashboard Capabilities

- Escrow list with filters (status, date range, brand, creator, campaign).
- Manual release/refund with mandatory reason capture.
- Dispute/chargeback queue (escrows currently `Disputed`).
- Fraud alerts feed (from velocity checks, §12.9).
- Webhook monitor — health/lag view of incoming Stripe events, surfaced from the `webhook_events` dedup table.
- Metrics: revenue (platform fee), total escrow value held, payout volume, refund rate, dispute count, conversion rate — all sourced from the analytics events in §18.

19.2 Audit Log Requirements

Every admin-privileged action (manual release, manual refund, any future manual state override) must write an immutable audit log entry containing:

- Admin identity (user ID + name)
- Action type
- Target entity (escrow/tip/payout ID)
- Before-state and after-state
- Timestamp
- Mandatory reason text
- Source IP (for security review)

Audit logs are append-only at the database layer — no UI or API path exists to edit or delete an entry.

19.3 Search & Filters

Admin escrow/audit views support search by brand name, creator name, campaign name, and transaction ID, plus filtering by status and date range, consistent with the Transaction History screen (9.10) filter patterns.

20. Engineering Status

(Carried forward from the original PRD, unchanged except as noted.)

Component	Status	Notes
Stripe Connect creator onboarding	✅ Complete	Express accounts with KYC
Escrow fund endpoint	✅ Complete	PaymentIntent with transfer_data
Escrow release endpoint	✅ Complete	Marks released, creates payout record
Escrow refund endpoint	✅ Complete	Full refund via Stripe API
Escrow status endpoint	✅ Complete	Get current status + amounts
Auto-release cron job	❌ Removed (2026-07-20)	Worker + hourly scan removed; release is brand-triggered only
Tips (send tip)	✅ Complete	Direct charge with immediate transfer
Payout history (creator)	✅ Complete	List all payouts with filtering
Webhook handler (11 events)	✅ Complete	Handles all critical Stripe events
Internal API for campaign-service	✅ Complete	Check escrow status by campaignId / inviteId
Notification integration	✅ Complete	All payment events trigger notifications (HTTP to notification-service; no BullMQ)

Component	Status	Notes
Database schema + migrations	✅ Complete	4 tables: escrow, tips, payouts, webhook events
Zod input validation	✅ Complete	All endpoints validated
Error handling	✅ Complete	Consistent error codes and messages
Idempotency	✅ Complete	On all Stripe operations
Dispute-created notification (<i>new gap identified in this spec</i>)	🕒 Not built	Needed to support Screen 20 / §15 addition
Admin mandatory-reason override flow (<i>new gap identified in this spec</i>)	🕒 Not built	Needed for §19.2 audit compliance
India GST line-item calculation	🕒 Pending	Blocked on Finance/Legal review (§21)

21. Go-Live Readiness

Requirement	Owner	Status
Live Stripe API keys (requires registered business)	Finance + DevOps	🕒 Pending
Production webhook endpoint registered in Stripe dashboard	DevOps	🕒 Pending
Frontend: Stripe Elements integration (card input UI)	Frontend Team	🕒 Pending
Frontend: Escrow status display on campaign pages	Frontend Team	🕒 Pending
Frontend: Creator earnings dashboard	Frontend Team	🕒 Pending
Frontend: Tip button on creator profiles	Frontend Team	🕒 Pending
Frontend: Dispute Center (Screen 20)	Frontend Team	🕒 Pending — new in this spec
Backend: Dispute-created notification	Engineering	🕒 Pending — new in this spec
Backend: Admin mandatory-reason override	Engineering	🕒 Pending — new in this spec
QA testing on Stripe test mode (full plan in §17)	QA Team	🕒 Pending
Legal review of fee structure and terms of service	Legal	🕒 Pending
GST / tax handling review (India-specific)	Finance	🕒 Pending
WCAG 2.2 AA audit across all 20 screens	Design + QA	🕒 Pending — new in this spec
Decision on auto-release reinstatement (see §22)	Product	🕒 Pending decision

22. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Brand's card is declined	Medium	Low	Clear error message with "try again" option; support multiple payment methods
Creator doesn't complete Stripe onboarding	Medium	Medium	Reminder emails at day 1, 3, 7; onboarding progress bar (Screen 1)
Stripe service outage	Very Low	High	Friendly error state; queue failed operations for retry where safe

Risk	Likelihood	Impact	Mitigation
Chargeback / dispute filed by brand	Low	Medium	Stripe handles evidence submission; auto-notify admin via webhook; Dispute Center (Screen 20) keeps both parties informed
Creator bank payout fails	Low	Medium	Stripe retries automatically; creator notified to update bank details
Regulatory change (tax, compliance)	Low	Medium	Modular fee calculation; GST addable as a separate line item
Trust erosion from removed auto-release <i>(new — flagged in this spec)</i>	Medium	Medium-High	Without a guaranteed release timer, a non-responsive brand can leave a creator's funds indefinitely Held . Mitigate short-term with proactive reminder notifications to brands with long- Held escrows and clear in-product messaging to creators that release is manual; evaluate Phase 2 reinstatement of an opt-in or safeguarded auto-release (e.g., with fraud checks the original blind timer lacked)
Notification silently lost on notification-service outage <i>(new — flagged in this spec)</i>	Low-Medium	Medium	HTTP-based (non-queued) integration means a notification-service outage during a payment event produces no retry queue; add either a retry-with-backoff wrapper or a lightweight persisted "pending notifications" fallback
Admin override without audit trail <i>(new — flagged in this spec, addressed in §19.2)</i>	Low	High (compliance/trust)	Mandatory reason + immutable audit log required before Admin override ships to production

23. Future Roadmap (Phase 2+)

Feature	Description	Priority
Milestone-based payments	Split campaign payment into stages (e.g., 50% on draft approval, 50% on publish)	High
Partial refunds	Refund a portion of the escrow (e.g., for partial delivery)	High
Multi-currency support	Support USD, EUR alongside GBP and INR	Medium
Automatic GST calculation	Calculate and display GST for Indian transactions	High (legal)
Creator dispute UI	Allow creators to dispute a refund request through the platform	Medium
Subscription payments	Monthly retainer campaigns with recurring billing	Low
Analytics dashboard	Revenue, payout, and dispute analytics for admins (builds on §18/§19 foundations)	Medium
Tip leaderboard	Public top tippers on creator profiles (gamification)	Low
In-app Dispute Resolution Center <i>(new — recommended in §10 benchmarking)</i>	Evidence submission and mediation inside Vitaay rather than routing entirely to Stripe's dashboard	Medium-High
Safeguarded auto-release reinstatement <i>(new — recommended in §22)</i>	Reintroduce a timer-based release with fraud/dispute safeguards the original design lacked	High

24. Glossary

Term	Plain English Explanation
Escrow	A safe holding area for money. The brand pays, the money is held by Stripe, and only released to the creator when the brand approves the deliverable.
Stripe	The payment processing company Vitaay uses. Handles all card charges, bank transfers, and regulatory compliance.
Stripe Connect	A Stripe product that lets Vitaay pay creators directly. Each creator gets their own Stripe sub-account.
PaymentIntent	Stripe's term for a single payment transaction. One escrow = one PaymentIntent.
Platform Fee	The 8% cut Vitaay takes from each transaction. This is how Vitaay makes money.
KYC	"Know Your Customer" — identity verification required by financial regulations. Stripe handles this during creator onboarding.
Webhook	Stripe sends notifications to Vitaay when something happens (payment succeeds, fails, disputed, etc.) — like a text message from Stripe to our server.
Idempotency	A safety mechanism that prevents the same payment from being charged twice, even on a double-click or network retry.
Auto-release	<i>(Removed 2026-07-20 — no longer implemented.)</i> Originally: if the brand didn't approve or reject within 5 days, the system would automatically pay the creator. Now a held escrow is released only by an explicit brand action.
Payout	When money moves from the creator's Stripe balance to their actual bank account. Stripe does this on a weekly schedule (every Friday).
Tip	A direct, voluntary payment from any user to a creator. Not tied to a campaign. Goes immediately to the creator.
SCA / 3D Secure	An extra authentication step (often a bank app confirmation) required for many EEA card payments under PSD2.
Chargeback / Dispute	When a cardholder's bank forcibly reverses a charge, usually after the cardholder contests it. Stripe manages evidence submission.
SAQ A	The lightest PCI-DSS self-assessment tier, available to merchants (like Vitaay) that fully outsource card handling to a compliant processor.

25. Appendices

Appendix A — Fee Breakdown Example (from original PRD, retained)

Line Item	Example (£500 campaign)
Campaign fee	£500.00
Transaction Charges	£40.00
Total charged to brand	£500.00
Creator receives	£460.00

Appendix B — Screen Inventory Summary

#	Screen	User	Priority
1	Creator Payment Onboarding	Creator	P0

#	Screen	User	Priority
2	Brand Checkout / Fund Escrow	Brand	P0
3	Stripe Payment (Element)	Brand / Any	P0
4	Processing Screen	Brand / Any	P0
5	Success Screen	Brand / Creator / Any	P0
6	Failure Screen	Brand / Any	P0
7	Escrow Status Card	Brand + Creator	P0
8	Release Payment Confirmation	Brand	P0
9	Refund Confirmation	Brand	P1
10	Transaction History	Brand / Creator / Admin	P0
11	Creator Earnings Dashboard	Creator	P0
12	Tip Button + Modal	Any User	P1
13	Brand Dashboard (Payments Widget)	Brand	P1
14	Notification Center	All	P1
15	Payment Settings	Creator	P1
16	Bank Settings (external)	Creator	P1
17	Admin Escrow Management	Admin	P2
18	Audit Logs	Admin	P1
19	Support Screen	Brand / Creator	P1
20	Dispute Center	Brand / Creator / Admin	P1

Appendix C — Original Payment Flow Diagrams (Preserved for History)

Auto-release path — **removed 2026-07-20**, retained for historical reference only:

```

Brand          Vitaay Platform          Stripe          Creator
|-- Fund Campaign ----->|              |              |
|                          |<-- Escrow: HELD -----|              |
|    ... 5 days pass, brand doesn't respond ...    |              |
|                          [Auto-Release Cron]      |              |
|                          |-- Transfer to Creator -->|              |
|                          |                          |-- Funds landed -->|
|<-- Escrow: AUTO-RELEASED -|              |              |

```

Appendix D — Notes on Source Document Fidelity

This document preserves every fact, table, and diagram from the original PRD v1.0 (including the descoped auto-release path, marked accordingly) and adds the sections and gap-flags called out throughout (search this document for “*new*” or “*gap identified*” markers) without removing any original content.